Master Angular 22 in 14 Days

A React / Next.js developer's plan, built around one real project: **RoyalStay CMS** · Prepared for Mahmoud As-Seyyid Ahmed · October 2026

How this plan works. You already know components, state, routing and data fetching from React and Next.js, so each day maps a React idea to its Angular 22 equivalent, then you apply it in RoyalStay CMS, a hotel admin panel connected to your own Express API (the-wild-oasis-express). By Day 14 you have a deployed full-stack Angular project for your CV.

Daily routine (about 2.5–3 hours)

Time	What to do
10 min	Review yesterday's notes and answer yesterday's *Check yourself* questions without looking.
40–60 min	Learn: read the day's angular.dev pages (they are the most up-to-date source for v22).
60–90 min	Build: complete the day's checklist in RoyalStay CMS.
15 min	Rebuild one piece from memory, then write notes: *React version → Angular version → one gotcha*.
5 min	Commit & push with a clear message; tick the day in the progress tracker.

Tip: many YouTube courses still teach older Angular (`@Input()`, `NgModule`, `*ngIf`, `ngOnChanges`). When you see those, translate them into the modern APIs in the cheat sheet below.

React / Next.js → Angular 22 cheat sheet

React / Next.js	Angular 22
useState	`signal()`
useMemo / derived	`computed()`
state reset on prop change	`linkedSignal()`
useEffect	`effect() (rarely) / afterRenderEffect()`
props	`input() / input.required()`
callback props	`output()`
controlled input	`model() + [(value)]`
children	`<ng-content>`
useRef to child	`viewChild()`
Context / Redux	`service + signals + inject()`
React Query / useFetch	`httpResource() / resource()`
{cond && ...}	`@if`
array.map + key	`@for (... ; track id)`

React.lazy + Suspense	`loadComponent + @defer`
React Router / app router	`provideRouter + <router-outlet>`
useParams	`route param as input()`
React Hook Form	`Signal Forms: form()`
Axios interceptor	`HttpInterceptorFn`
middleware / protected route	`CanActivateFn guard`
Jest + RTL	`Vitest + TestBed`

Progress tracker

Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7
Day 8	**Day 9**	**Day 10**	**Day 11**	**Day 12**	**Day 13**	**Day 14**

The 14 days at a glance

Day	Topic	You'll finish with
Day 1	Modern Angular 22: Setup & Mindset	The app runs with a sidebar + header layout and is on GitHub.
Day 2	Components & Templates	The Rooms page shows a styled table of mock rooms with an empty state.
Day 3	Signals: State, Derived State & Effects	Filtering and sorting work entirely with signals.
Day 4	Component Communication	Filter, modal and confirm dialog are reusable components.
Day 5	Dependency Injection & Services (Shared State)	Rooms state lives in a service and dark mode works app-wide.
Day 6	Routing	Every page has its own lazy route and the filter survives a refresh.
Day 7	HTTP & Resources (Real Data)	The Rooms pages show live data from your own backend.
Day 8	RxJS Essentials & Signal Interop	Guest search is debounced, cancels stale requests and never leaks.
Day 9	Signal Forms I (stable in v22)	Rooms can be created and edited through a validated form.
Day 10	Signal Forms II + Authentication	Only logged-in users can reach the app; requests carry the JWT.
Day 11	Performance, @defer & Reusable Logic	Dashboard loads fast with deferred charts; one pipe and one directive written.
Day 12	Testing with Vitest	At least 6 passing tests covering store, component, HTTP, form and guard.

Day 13	Accessibility, RTL & Polish	Keyboard-friendly, RTL-ready app with a Lighthouse accessibility score of 90+.
Day 14	Ship It & Show It	Live link on your CV and LinkedIn - you have a full-stack Angular project.

Always-useful resources

- Learn Angular interactive tutorial – quick hands-on refresh in the browser
- Angular Essentials – the core concepts in one place
- Angular style guide – naming and structure conventions interviewers expect
- Migrations reference – translate old Angular code you meet at work into modern APIs
- Angular Blog – official release announcements
- Angular AI page – the Angular MCP server and AI-assisted development (many teams now expect this)

Day 1 Modern Angular 22: Setup & Mindset

Goal: Understand how Angular 22 apps are structured and what changed from older tutorials, then scaffold RoyalStay CMS.

React bridge: Angular CLI ≈ Vite/create-next-app, but with built-in routing, HTTP, forms and testing. Everything is standalone (no NgModules) - think of each component as a self-contained React component that imports what it uses.

KEY CONCEPTS

- Standalone components (no NgModule)
- Angular 22 defaults: OnPush change detection, zoneless, strict templates, Vitest
- Workspace & file structure, angular.json
- Style guide naming (user-profile.ts, no .component suffix)

READ & WATCH

- What is Angular
  angular.dev/overview
- Essentials (start here)
  angular.dev/essentials
- Local setup & CLI
  angular.dev/tools/cli/setup-local
- Workspace file structure
  angular.dev/reference/configs/file-structure
- Official style guide
  angular.dev/style-guide
- What's new in Angular 22 (Ninja Squad)
  blog.ninja-squad.com/2026/06/03/what-is-new-angular-22.0
- Announcing Angular v22 (Angular Blog)
  blog.angular.dev/announcing-angular-v22-c52bb83a4664

TODAY'S CHECKLIST

- [ ] Read Essentials overview + Ninja Squad v22 summary (note every v22 default)
- [ ] Install/upgrade the CLI and run ng new royalstay-cms (SCSS or Tailwind, no SSR yet)
- [ ] Add Tailwind if you like it (guide/tailwind)
- [ ] Create the shell: AppLayout, Sidebar, Header components with ng generate
- [ ] Explore the generated files and write 1 line on what each does
- [ ] Push the repo to GitHub with a README that says what the project is
- [ ] Rebuild one piece from memory & write React → Angular notes
- [ ] Commit & push

CHECK YOURSELF (ANSWER OUT LOUD)

- [ ] What does OnPush-by-default mean for when a component re-renders?
- [ ] Why don't new Angular apps need NgModules?

Done when: The app runs with a sidebar + header layout and is on GitHub.

Notes / gotchas:

Day 2 Components & Templates

Goal: Write components fluently: bindings, events, control flow, pipes and styling.

React bridge: JSX {cond && ...} → @if, array.map → @for (track is the React key), className/style → [class]/[style] bindings, onClick → (click).

KEY CONCEPTS

- Property, attribute, class & style binding
- Event listeners
- @if / @else, @for with track and @empty, @switch, @let
- Built-in pipes (date, currency, async) and component styling/encapsulation

READ & WATCH

- Essentials: templates
  angular.dev/essentials/templates
- Components guide
  angular.dev/guide/components
- Binding
  angular.dev/guide/templates/binding
- Event listeners
  angular.dev/guide/templates/event-listeners
- Control flow
  angular.dev/guide/templates/control-flow
- @let & template variables
  angular.dev/guide/templates/variables
- Pipes
  angular.dev/guide/templates/pipes
- Component styling
  angular.dev/guide/components/styling

TODAY'S CHECKLIST

- [ ] Create a Room model (TypeScript interface) and a mock rooms array
- [ ] Build RoomRow and RoomsTable components rendering the list with @for + track
- [ ] Show an empty state with @empty and a discount badge with @if
- [ ] Format price with the currency pipe and dates with the date pipe
- [ ] Style the table with scoped component styles
- [ ] Rebuild RoomRow once from memory
- [ ] Rebuild one piece from memory & write React → Angular notes
- [ ] Commit & push

CHECK YOURSELF (ANSWER OUT LOUD)

- [ ] Why does @for require track, and what should you track by?
- [ ] What is view encapsulation?

Done when: The Rooms page shows a styled table of mock rooms with an empty state.

Notes / gotchas:

Day 3 Signals: State, Derived State & Effects

Goal: Manage local state the Angular 22 way.

React bridge: useState → signal(), useMemo → computed(), state reset when a prop changes → linkedSignal(), useEffect → effect() (but use it far less than useEffect).

KEY CONCEPTS

- signal(), set(), update()
- computed() and dependency tracking
- linkedSignal() for writable derived state
- effect() and when NOT to use it; debounced() (new in v22, experimental)

READ & WATCH

- Essentials: signals
  angular.dev/essentials/signals
- Signals guide
  angular.dev/guide/signals
- linkedSignal
  angular.dev/guide/signals/linked-signal
- Effects
  angular.dev/guide/signals/effect
- debounced() (v22)
  angular.dev/guide/signals/debounced

TODAY'S CHECKLIST

- [ ] Add filter (all / with discount / no discount) and sort signals to the Rooms page
- [ ] Create a computed filteredRooms() and render it instead of the raw list
- [ ] Use linkedSignal so the selected room resets when the list changes
- [ ] Write one effect that saves the filter to localStorage, then ask: could computed do this instead?
- [ ] Write a React vs Angular snippet comparison in your notes
- [ ] Rebuild one piece from memory & write React → Angular notes
- [ ] Commit & push

CHECK YOURSELF (ANSWER OUT LOUD)

- [ ] Why is computed() better than effect() for derived values?
- [ ] What makes a computed re-run?

Done when: Filtering and sorting work entirely with signals.

Notes / gotchas:

Day 4 Component Communication

Goal: Pass data down, events up, and build reusable wrapper components.

React bridge: props → input(), callback props → output(), controlled component → model() + [( )], children → <ng-content>, useRef to a child → viewChild().

KEY CONCEPTS

- input() and input.required()
- output() and emitting events
- model() two-way binding
- Content projection with ng-content and queries (viewChild, contentChild)

READ & WATCH

- Inputs
  angular.dev/guide/components/inputs
- Outputs
  angular.dev/guide/components/outputs
- Two-way binding
  angular.dev/guide/templates/two-way-binding
- Content projection
  angular.dev/guide/components/content-projection
- Queries
  angular.dev/guide/components/queries
- Lifecycle
  angular.dev/guide/components/lifecycle

TODAY'S CHECKLIST

- [ ] Extract a RoomsFilter component that receives the current value (input) and emits changes (output)
- [ ] Convert it to model() and compare the two approaches
- [ ] Build a reusable Modal component using ng-content
- [ ] Build a ConfirmDelete dialog that emits confirm/cancel
- [ ] Use viewChild to focus the first input when the modal opens
- [ ] Rebuild one piece from memory & write React → Angular notes
- [ ] Commit & push

CHECK YOURSELF (ANSWER OUT LOUD)

- [ ] When would you choose model() over input()+output()?
- [ ] What is the Angular equivalent of props.children?

Done when: Filter, modal and confirm dialog are reusable components.

Notes / gotchas:

Day 5 Dependency Injection & Services (Shared State)

Goal: Move shared logic and state into services.

React bridge: Context + custom hooks / Redux store → a service with signals, injected with inject(). providedIn: 'root' ≈ a global store; component providers ≈ scoped context.

KEY CONCEPTS

- inject() function
- @Injectable({providedIn: 'root'}) and the new @Service decorator (v22)
- Hierarchical injectors & scoping
- Signal-based store pattern (private writable signal, public readonly + methods)

READ & WATCH

- Essentials: DI
  angular.dev/essentials/dependency-injection
- DI overview
  angular.dev/guide/di
- Creating & using services
  angular.dev/guide/di/creating-and-using-services
- Hierarchical DI
  angular.dev/guide/di/hierarchical-dependency-injection
- @Injectable → @Service migration
  angular.dev/reference/migrations/injectable-to-service
- NgRx SignalStore (bonus)
  ngrx.io/guide/signals/signal-store

TODAY'S CHECKLIST

- [ ] Create RoomsStore: rooms signal, selectedRoom, add/update/delete methods
- [ ] Expose state as readonly signals (asReadonly())
- [ ] Make the Rooms page and Modal use the store instead of local state
- [ ] Create ThemeService for dark mode (signal + localStorage)
- [ ] Try providing a service at component level and observe the separate instance
- [ ] Rebuild one piece from memory & write React → Angular notes
- [ ] Commit & push

CHECK YOURSELF (ANSWER OUT LOUD)

- [ ] What's the difference between providedIn: 'root' and providers: [] on a component?
- [ ] Why expose readonly signals from a store?

Done when: Rooms state lives in a service and dark mode works app-wide.

Notes / gotchas:

Day 6 Routing

Goal: Build multi-page navigation with lazy loading and route params.

React bridge: React Router / Next.js app router → provideRouter + routes array. <Outlet/> → <router-outlet>, <Link> → routerLink, useParams → input() via withComponentInputBinding(), useSearchParams → queryParams.

KEY CONCEPTS

- Defining routes, redirects, wildcard 404
- Lazy loading with loadComponent / loadChildren
- Nested routes & outlets
- Reading params & query params; programmatic navigation

READ & WATCH

- Routing overview
  angular.dev/guide/routing
- Define routes
  angular.dev/guide/routing/define-routes
- Outlets
  angular.dev/guide/routing/show-routes-with-outlets
- Navigation
  angular.dev/guide/routing/navigate-to-routes
- Read route state
  angular.dev/guide/routing/read-route-state
- Lazy loading strategies
  angular.dev/guide/routing/loading-strategies
- Redirects
  angular.dev/guide/routing/redirecting-routes

TODAY'S CHECKLIST

- [ ] Routes: /dashboard, /rooms, /rooms/:id, /bookings, /guests, /settings, /login, ** (404)
- [ ] Lazy-load every page with loadComponent
- [ ] Enable withComponentInputBinding() and read :id as an input on RoomDetails
- [ ] Store the rooms filter in the URL as a query param (shareable links)
- [ ] Highlight the active sidebar link with routerLinkActive
- [ ] Rebuild one piece from memory & write React → Angular notes
- [ ] Commit & push

CHECK YOURSELF (ANSWER OUT LOUD)

- [ ] How do you read a route param as a component input?
- [ ] Why lazy-load routes?

Done when: Every page has its own lazy route and the filter survives a refresh.

Notes / gotchas:

Day 7 HTTP & Resources (Real Data)

Goal: Load real data from your Express API with Angular 22's stable resource APIs.

React bridge: fetch + useEffect / React Query → httpResource() (signals in, data/loading/error signals out). Mutations still use HttpClient.

KEY CONCEPTS

- provideHttpClient() setup
- HttpClient get/post/patch/delete (Observables)
- httpResource() and resource(): value, isLoading, error, reload
- Data fetching with resources in routes

READ & WATCH

- HTTP setup
  angular.dev/guide/http/setup
- Making requests
  angular.dev/guide/http/making-requests
- httpResource
  angular.dev/guide/http/http-resource
- resource()
  angular.dev/guide/signals/resource
- Routing + resources
  angular.dev/guide/routing/data-fetching-with-resources

TODAY'S CHECKLIST

- [ ] Run your the-wild-oasis-express API (locally or the Vercel URL) and put the base URL in an environment file
- [ ] Load cabins/rooms with httpResource() and show loading + error states
- [ ] Make the room details page fetch by :id using a resource that depends on the id input
- [ ] Create/update/delete rooms with HttpClient, then reload() the resource
- [ ] Week-1 review: list 10 React → Angular mappings from memory
- [ ] Rebuild one piece from memory & write React → Angular notes
- [ ] Commit & push

CHECK YOURSELF (ANSWER OUT LOUD)

- [ ] What signals does httpResource give you?
- [ ] Why do mutations use HttpClient instead of httpResource?

Done when: The Rooms pages show live data from your own backend.

Notes / gotchas:

Day 8 RxJS Essentials & Signal Interop

Goal: Learn the RxJS you'll meet in real Angular codebases and job interviews.

React bridge: No direct React equivalent: Observables are streams of values over time. Think of them as event emitters with powerful operators. Signals hold the current value; Observables model events.

KEY CONCEPTS

- Observable, subscribe, unsubscribe
- Operators: map, filter, tap, debounceTime, distinctUntilChanged, switchMap, catchError
- Avoiding leaks with takeUntilDestroyed()
- toSignal(), toObservable(), rxResource()

READ & WATCH

- RxJS official guide
  rxjs.dev/guide/overview
- Learn RxJS (operator examples)
  www.learnrxjs.io/
- RxMarbles (visual operators)
  rxmarbles.com/
- RxJS interop with signals
  angular.dev/ecosystem/rxjs-interop
- takeUntilDestroyed
  angular.dev/ecosystem/rxjs-interop/take-until-destroyed

TODAY'S CHECKLIST

- [ ] Read the RxJS overview and play with 5 operators on RxMarbles
- [ ] Build a debounced guest search: input → debounceTime → distinctUntilChanged → switchMap(http)
- [ ] Convert the result to a signal with toSignal()
- [ ] Rebuild the same search with the v22 debounced() signal and compare
- [ ] Find one subscription you wrote and make it leak-proof with takeUntilDestroyed()
- [ ] Rebuild one piece from memory & write React → Angular notes
- [ ] Commit & push

CHECK YOURSELF (ANSWER OUT LOUD)

- [ ] Why switchMap (not mergeMap) for search?
- [ ] When would you keep an Observable instead of converting to a signal?

Done when: Guest search is debounced, cancels stale requests and never leaks.

Notes / gotchas:

Day 9 Signal Forms I (stable in v22)

Goal: Build typed, validated forms with Angular's new Signal Forms.

React bridge: React Hook Form / Formik → form(model, schema). The form state (value, valid, touched, errors) is all signals.

KEY CONCEPTS

- Designing the form model
- form() and binding fields with [field]
- Built-in validators: required, min/max, minLength, minDate/maxDate (v22)
- Field state (touched, dirty, errors) and submission

READ & WATCH

- Essentials: Signal Forms
  angular.dev/essentials/signal-forms
- Signal Forms overview
  angular.dev/guide/forms/signals/overview
- Form models
  angular.dev/guide/forms/signals/models
- Validation
  angular.dev/guide/forms/signals/validation
- Field state
  angular.dev/guide/forms/signals/field-state-management
- Submission
  angular.dev/guide/forms/signals/form-submission
- Signal vs Reactive forms
  angular.dev/guide/forms/signals/comparison

TODAY'S CHECKLIST

- [ ] Build the Create/Edit Room form (name, capacity, price, discount, description, image)
- [ ] Add validation: required fields, capacity 1-10, discount ≤ price
- [ ] Show error messages only after a field is touched
- [ ] Submit with HttpClient (POST for new, PATCH for edit) and close the modal
- [ ] Read the Signal vs Reactive forms comparison so you can talk about both in interviews
- [ ] Rebuild one piece from memory & write React → Angular notes
- [ ] Commit & push

CHECK YOURSELF (ANSWER OUT LOUD)

- [ ] How do you show an error only after the user leaves a field?
- [ ] When might a company still use Reactive Forms?

Done when: Rooms can be created and edited through a validated form.

Notes / gotchas:

Day 10 Signal Forms II + Authentication

Goal: Handle cross-field and async validation, then secure the app with JWT.

React bridge: Axios interceptors → HttpInterceptorFn. Next.js middleware / protected routes → functional route guards (canActivate).

KEY CONCEPTS

- Cross-field logic & async validation
- Functional HTTP interceptors
- Functional route guards (CanActivateFn)
- Auth service storing the token and user as signals

READ & WATCH

- Cross-field logic
  angular.dev/guide/forms/signals/cross-field-logic
- Async operations
  angular.dev/guide/forms/signals/async-operations
- Interceptors
  angular.dev/guide/http/interceptors
- Route guards
  angular.dev/guide/routing/route-guards
- Security best practices
  angular.dev/best-practices/security

TODAY'S CHECKLIST

- [ ] Build the Login form (email + password) against your API's /users/login
- [ ] AuthService: token + currentUser signals, login(), logout(), persist token
- [ ] authInterceptor adds 'Authorization: Bearer <token>' to every request
- [ ] authGuard redirects to /login when there's no token; admin-only guard for Settings
- [ ] Sign-up form with password + confirm password (cross-field validation)
- [ ] Rebuild one piece from memory & write React → Angular notes
- [ ] Commit & push

CHECK YOURSELF (ANSWER OUT LOUD)

- [ ] Where does the interceptor get the token from?
- [ ] What's the difference between canActivate and canMatch?

Done when: Only logged-in users can reach the app; requests carry the JWT.

Notes / gotchas:

Day 11 Performance, @defer & Reusable Logic

Goal: Make the app fast and learn directives and custom pipes.

React bridge: React.lazy + Suspense → @defer blocks. React.memo is mostly unnecessary: OnPush + signals already limit re-rendering. Custom hooks for DOM behaviour → attribute directives.

KEY CONCEPTS

- @defer with triggers (viewport, idle, interaction) and placeholders
- Zoneless change detection and OnPush
- Custom pipes and attribute directives
- Error boundaries with @boundary (developer preview)

READ & WATCH

- Deferrable views (@defer)
  angular.dev/guide/templates/defer
- Zoneless
  angular.dev/guide/zoneless
- Runtime performance
  angular.dev/best-practices/runtime-performance
- Attribute directives
  angular.dev/guide/directives/attribute-directives
- Custom pipes
  angular.dev/guide/templates/pipes
- Error boundaries
  angular.dev/guide/templates/error-boundaries

TODAY'S CHECKLIST

- [ ] Build the Dashboard: stat cards (bookings, sales, occupancy) from your API
- [ ] Load the charts section with @defer (on viewport) + a skeleton placeholder
- [ ] Write a custom pipe (e.g. nights between two dates)
- [ ] Write an attribute directive (e.g. appAutofocus or a status-colour highlight)
- [ ] Profile the app with Angular DevTools and note one improvement
- [ ] Rebuild one piece from memory & write React → Angular notes
- [ ] Commit & push

CHECK YOURSELF (ANSWER OUT LOUD)

- [ ] What triggers does @defer support?
- [ ] Why does OnPush + signals reduce the need for memoization?

Done when: Dashboard loads fast with deferred charts; one pipe and one directive written.

Notes / gotchas:

Day 12 Testing with Vitest

Goal: Close the testing gap from your job applications.

React bridge: Jest + React Testing Library → Vitest + TestBed (or Angular Testing Library). Same ideas: render, interact, assert.

KEY CONCEPTS

- Vitest as Angular's default runner
- Testing components with TestBed
- Testing services and HTTP with HttpTestingController
- Testing Signal Forms and routes

READ & WATCH

- Testing overview
  angular.dev/guide/testing
- Component testing basics
  angular.dev/guide/testing/components-basics
- Testing services
  angular.dev/guide/testing/services
- Testing HTTP
  angular.dev/guide/http/testing
- Testing Signal Forms
  angular.dev/guide/forms/signals/testing
- Testing routing
  angular.dev/guide/routing/testing
- Vitest docs
  vitest.dev/guide/
- Angular Testing Library
  testing-library.com/docs/angular-testing-library/intro/

TODAY'S CHECKLIST

- [ ] Run ng test and read how Vitest is configured
- [ ] Test RoomsStore: add, update, delete change the signals correctly
- [ ] Test the RoomsTable: renders rows and the empty state
- [ ] Test AuthService login with HttpTestingController
- [ ] Test the Room form: invalid when required fields are empty
- [ ] Test authGuard redirects when logged out
- [ ] Rebuild one piece from memory & write React → Angular notes
- [ ] Commit & push

CHECK YOURSELF (ANSWER OUT LOUD)

- [ ] What does HttpTestingController let you do?
- [ ] What should a good component test assert?

Done when: At least 6 passing tests covering store, component, HTTP, form and guard.

Notes / gotchas:

Day 13 Accessibility, RTL & Polish

Goal: Make the app accessible and ready for Arabic users - both valued by Egyptian and Gulf employers.

React bridge: Headless UI / Radix → Angular Aria (stable in v22): accessible patterns like tabs, menus and selects with keyboard support built in.

KEY CONCEPTS

- Angular Aria patterns (tabs, menu, select, listbox)
- Accessibility best practices: labels, focus, keyboard navigation
- Right-to-left layout with dir='rtl' and logical CSS properties
- Error handling across the app

READ & WATCH

- Angular Aria overview
  angular.dev/guide/aria/overview
- Aria tabs
  angular.dev/guide/aria/tabs
- Aria menu
  angular.dev/guide/aria/menu
- Accessibility best practices
  angular.dev/best-practices/a11y
- Internationalization (i18n)
  angular.dev/guide/i18n
- Error handling
  angular.dev/best-practices/error-handling

TODAY'S CHECKLIST

- [ ] Use Aria tabs on the Settings page and an Aria menu for the user dropdown
- [ ] Check every form input has a label and every button is reachable by keyboard
- [ ] Add an Arabic/English toggle that switches dir='rtl' and swap margin-left/right for logical properties
- [ ] Add friendly error and empty states on every data page
- [ ] Run Lighthouse accessibility audit and fix the top issues
- [ ] Rebuild one piece from memory & write React → Angular notes
- [ ] Commit & push

CHECK YOURSELF (ANSWER OUT LOUD)

- [ ] What does Angular Aria give you that plain HTML doesn't?
- [ ] What are CSS logical properties?

Done when: Keyboard-friendly, RTL-ready app with a Lighthouse accessibility score of 90+.

Notes / gotchas:

Day 14 Ship It & Show It

Goal: Deploy RoyalStay CMS and turn it into a portfolio and CV asset.

React bridge: Same as deploying your Next.js and React apps to Vercel: build, set environment variables, deploy.

KEY CONCEPTS

- Production builds and environments
- Deployment (Vercel / Netlify / Firebase)
- Writing a strong README
- What to learn next: SSR & hydration, NgRx SignalStore

READ & WATCH

- Building the app
  angular.dev/tools/cli/build
- Environments
  angular.dev/tools/cli/environments
- Deployment
  angular.dev/tools/cli/deployment
- Server-side rendering (next step)
  angular.dev/guide/ssr
- Angular update guide
  angular.dev/update-guide

TODAY'S CHECKLIST

- [ ] ng build and fix any warnings
- [ ] Deploy to Vercel with the API URL as an environment variable
- [ ] README: features, tech stack, screenshots, live link, how to run
- [ ] Add RoyalStay CMS to your CV (Angular jobs version), LinkedIn Projects and pin it on GitHub
- [ ] Re-answer every 'Check yourself' question from Days 1-13 out loud
- [ ] Pick your next topic: SSR or NgRx SignalStore
- [ ] Rebuild one piece from memory & write React → Angular notes
- [ ] Commit & push

CHECK YOURSELF (ANSWER OUT LOUD)

- [ ] Can you explain your app's architecture in 2 minutes?
- [ ] What would you improve next and why?

Done when: Live link on your CV and LinkedIn - you have a full-stack Angular project.

Notes / gotchas: